

Examen Final: Estructuras de Datos 2024

Tema Central: Universo Saga Cinematográfica de Matrix

Instrucciones Generales: Responde todas las preguntas en base al contexto presentado utilizando exclusivamente Python para las partes prácticas.

Parte Teórica

1. Encapsulamiento e Interfaces

- Neo encuentra un programa encapsulado que controla la entrada y salida de personas a la Matrix. Define qué es una interfaz en el contexto de la Programación Orientada a Objetos y explica cómo podría usarse para representar este programa.

2. Estructuras de Datos Recursivas

- Considera que las conexiones dentro de la Matrix pueden representarse como un grafo recursivo. Define un grafo y describe dos representaciones posibles. ¿Cuándo es preferible usar listas de adyacencia en lugar de matrices de adyacencia?

3. Análisis de Algoritmos

- Trinity implementa un algoritmo para hackear un sistema en la Matrix. Explica qué es la notación $O()$ y cómo se aplica para determinar el mejor, peor y promedio caso de un algoritmo de búsqueda dentro de un arreglo desordenado.

4. Problemas NP

- Morpheus intenta encontrar el camino más corto para alcanzar un terminal dentro de la Matrix. Explica por qué el problema del camino más corto es considerado NP-completo y describe el uso del algoritmo de Dijkstra para resolverlo.
-

Parte Práctica

1. Árboles Binarios

- Representa el proceso de decisión de Neo para elegir entre la píldora roja y la azul como un árbol binario. Implementa en Python un árbol con los nodos "Inicio", "Píldora Roja" y "Píldora Azul". Realiza un recorrido InOrden del árbol e imprime los nodos visitados.

2. Heap Binaria y Cola de Prioridades

- El oráculo gestiona una cola de prioridad de eventos futuros en la Matrix. Implementa una estructura de Heap Binaria para gestionar los eventos según su importancia (prioridad). Incluye las operaciones de inserción y extracción del máximo.

3. Algoritmos de Recorrido

- Diseña una función en Python para realizar un recorrido BFS en un grafo que representa las conexiones de la Matrix. Usa el siguiente conjunto de datos para el grafo:
 - Nodos: [A, B, C, D, E]
 - Aristas: [(A, B), (A, C), (B, D), (C, E), (D, E)]

4. Ordenamiento Topológico

- El Arquitecto desea ordenar las dependencias entre programas para reconstruir la Matrix. Implementa un algoritmo de ordenamiento topológico en Python para el siguiente grafo dirigido:
 - Nodos: [A, B, C, D, E]
 - Aristas: [(A, B), (A, C), (B, D), (C, D), (D, E)]

5. Heap Binario para Planificación

- Smith utiliza un montículo binario (heap) para gestionar las tareas prioritarias en tiempo real. Implementa un heap mínimo en Python con soporte para operaciones de inserción, extracción del mínimo y disminución de clave. Prueba tu implementación con la siguiente lista de tareas y prioridades: [("Tarea A", 5), ("Tarea B", 3), ("Tarea C", 8), ("Tarea D", 1)].

6. Ruta Mínima en Grafos

- Neo necesita encontrar la ruta más corta para llegar a un terminal en un grafo ponderado que representa conexiones en la Matrix. Implementa el algoritmo de Dijkstra en Python y úsalo con el siguiente grafo:
 - Nodos: [1, 2, 3, 4, 5]
 - Aristas y pesos: [(1, 2, 2), (1, 3, 4), (2, 3, 1), (2, 4, 7), (3, 5, 3), (4, 5, 1)]

Criterios de Evaluación:

- Completitud y claridad de las respuestas.
- Correcta aplicación de los conceptos teóricos.
- Precisión y eficiencia en las implementaciones prácticas.
- Uso correcto de Python.

Duración del Examen: 3 horas.

Material Permitido: Apuntes y libros indicados en la bibliografía obligatoria del programa.

¡Buena suerte! "La elección, como siempre, está en tus manos."